

SQL

Language

- Set operations to define and manipulate structure and constraints of database.
- All related to relational model
- Formal Language → Relational Algebra, Relational Calculus
- SQL → success in business of DBMS tool

History

- SQL stand for **Structured Query Language**
- SQL is based on the Relational Tuple Calculus
- Evolved from SEQUEL: **S**tructured **E**nglish **Q**Uery **L**anguage - part of IBM's SYSTEM R, 1974
- SQL2 Supported by
 - ORACLE, SYBASE, INFORMIX,
 - IBM DB2, SQL SERVER, ...
 - MS Access, MySQL, ...
- SQL2 also called SQL/92 is evolved from SQL/86, SQL/89, all were ANSI & ISO standard
- Currently Working on SQL3/SQL-99 with OO Extensions
- Now – SQL is standard language for commercial relational DBMS

SQL Component

- **Data Definition Language (DDL)**
 - For External and Conceptual Schemas
 - Views - DDL for External Schemas
- **Data Manipulation Language (DML)**
 - Interactive DML Against External and Conceptual Schemas
 - Embedded DML in Host PLs (EQL, JDBC, etc.)
- **Others**
 - Integrity (Allowable Values/Referential)
 - Catalog and Dictionary Facilities
 - Transaction Control (Long-Duration and Batch)
 - Authorization (Who can Do What When)

Data Type

- **Numeric**
 - INTEGER (or INT), SMALLINT
 - REAL, DOUBLE PRECISION
 - FLOAT(N) Floating Point with at Least N Digits
 - DECIMAL(P,D) (DEC(P,D) or NUMERIC(P,D)) have P Total Digits with D to Right of Decimal
- Note that INTs and REALs are Machine Dependent (Based on Hardware/OS Platform)

Data Type

Character-String

- CHAR(N) or CHARACTER(N) – Fixed
- VARCHAR(N), CHAR VARYING(N), or CHARACTER VARYING(N)
Variable with at Most N Characters
- TEXT(N)

UniCode

- NCHAR(N)– Fixed
- NVARCHAR(N)– Fixed
- Text(N)– Fixed

Data Type

- Has DATE, TIME, and TIMESTAMP data types
 - **DATE:**
Made up of year-month-day in the format yyyy-mm-dd
 - **TIME:**
Made up of hour:minute:second in the format hh:mm:ss
 - **TIME(i):**
Made up of hour:minute:second plus i additional digits specifying fractions of a second
format is hh:mm:ss:ii...i
 - **TIMESTAMP:**
Has both DATE and TIME components

Data Type

Biner

- **Binary**
- **Varbinary**
- **Blob**

Others

- **Json**
- **XML**

Data Definition Language (DDL)

Data Definition Language (DDL)

Used to CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database.

STUDENT		
Name	<u>Ssn</u>	GPA

COURSE			
<u>Course_num</u>	Title	Description	<u>Pre_Course_num</u>

CREATE TABLE Student:
Name(CHAR(30)),SSN(CHAR(9)),GPA(FLOAT(2))

CREATE TABLE Courses:
Course#(CHAR(6)), Title(CHAR(20)), Descrip(CHAR(100)),
Pcourse#(CHAR(6))

SQL Schema

- SQL Schema is identified by **schema name and include authorization identifier.**
- **Schema elements:** tables, attributes names, constraints, views, domains and other construct (such as authorization grant) that describe the schema
- System Administrator or DBA had privilege to create schemas
- Features that added to SQL2 & SQL-99

Create/Drop Schema

- Creating a Schema:

CREATE SCHEMA FATISDA AUTHORIZATION BAMZ;

- Schema FATISDA has Been Created and is Owner by the User “BAMZ”
- Tables can now be Created and Added to Schema

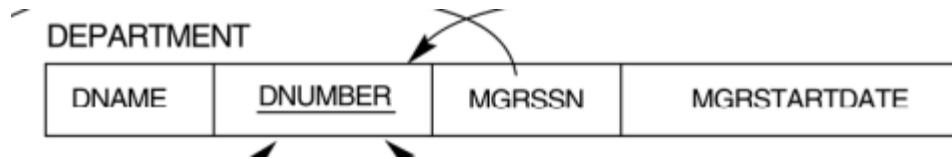
- Dropping a Schema:

DROP SCHEMA FATISDA RESTRICT;

DROP SCHEMA FATISDA CASCADE;

- Restrict: Drop Operation Fails If Schema is Not Empty
- Cascade: Drop Operation Removes Everything in the Schema

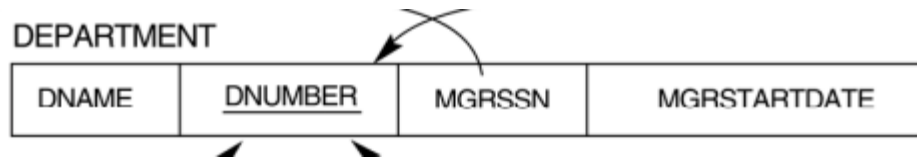
Create Table



- **Specifies a new base relation** by giving it a name, and specifying each of its attributes and their data types (INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n))
- **A constraint** NOT NULL may be specified on an attribute

```
CREATE TABLE DEPARTMENT
(
    DNAME    VARCHAR(10)    NOT NULL,
    DNUMBER  INTEGER        NOT NULL,
    MGRSSN   CHAR(9),
    MGRSTARTDATE CHAR(9) );
```

Create Table



- In SQL2, can use the CREATE TABLE command for specifying the primary key attributes, secondary keys, and referential integrity constraints (foreign keys).
- Key attributes can be specified via the PRIMARY KEY and UNIQUE phrases

```
CREATE TABLE DEPT
(
    DNAME    VARCHAR(10)    NOT NULL,
    DNUMBER  INTEGER        NOT NULL,
    MGRSSN   CHAR(9),
    MGRSTARTDATE CHAR(9),
    PRIMARY KEY (DNUMBER),
    UNIQUE (DNAME),
    FOREIGN KEY (MGRSSN) REFERENCES EMP );
```

Drop Table

- Used **to remove a relation** (base table) *and its definition*
- The relation can no longer be used in queries, updates, or any other commands since its description no longer exists
- Example:

DROP TABLE DEPARTMENT;

Alter Table

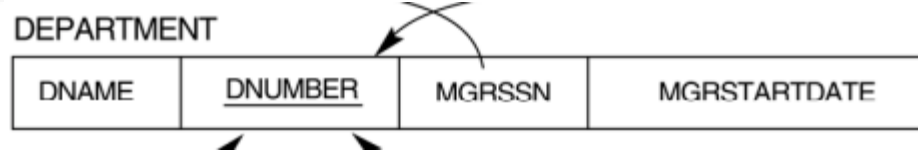
Used to add or Modify an attribute to one of the base relations

- The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is *not allowed* for such an attribute
- Example:

```
ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);
```

- The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple. This can be done using the UPDATE command.

Alter Table (2)



❖ Add Column

`ALTER TABLE table_name ADD column_name datatype;`

`ALTER TABLE DEPARTMENT ADD LASTUPDATE DATETIME NOT NULL`

❖ Modify DataType Column

`ALTER TABLE table_name ALTER COLUMN column_name datatype;`

❖ Rename Column

`ALTER TABLE table_name RENAME COLUMN old_name to new_name;`

❖ Drop Column

`ALTER TABLE table_name DROP COLUMN column_name;`

- ❖ **PRESENT DDL ON YOUR TEAM FINAL PROJECT**

Data Manipulation Language (DML)

Retrieve Queries in SQL (Select)

- SQL has **one basic statement** for retrieving information from a database; the **SELECT** statement
- Important distinction between SQL and the formal relational model; SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values
- Hence, an SQL relation (table) is a **multi-set** (sometimes called a bag) of tuples; it *is not* a set of tuples
- SQL relations **can be constrained** to be sets by specifying PRIMARY KEY or UNIQUE attributes, or by using the DISTINCT option in a query

Basic Retrieve Queries in SQL (USE OF *)

- To **retrieve all the attribute values** of the selected tuples, a ***** is used, which stands for *all the attributes*

Examples:

Q1a : SELECT *
 FROM EMPLOYEE

Q1a : SELECT *
 FROM DEPARTMENT

UNSPECIFIED WHERE-clause

- *A missing WHERE-clause indicates no condition*; hence, *all tuples* of the relations in the FROM-clause are selected
- This is equivalent to the condition **WHERE TRUE**
- Query 2: Retrieve the SSN values for all employees.

**Q2: SELECT SSN
FROM EMPLOYEE**

- If more than one relation is specified in the FROM-clause *and* there is no join condition, then the *CARTESIAN PRODUCT* of tuples is selected

Using SPECIFIED WHERE-clause

- To **retrieve all the attribute values** of the selected tuples, a ***** is used, which stands for *all the attributes*

Examples:

Q3 : SELECT *
 FROM EMPLOYEE
 WHERE DNO=5

Q4 : SELECT *
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNAME='Research' AND
 DNO=DNUMBER

Retrieve Queries in SQL (2)

- Basic SQL queries correspond to using the SELECT, PROJECT, and JOIN operations of the relational algebra
- All subsequent examples use the COMPANY database
- Example of a simple query on *one* relation
- Query 5: Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

Q5:

```

SELECT      BDATE, ADDRESS
FROM        EMPLOYEE
WHERE       FNAME='John' AND MINIT='B'
AND         LNAME='Smith'
  
```

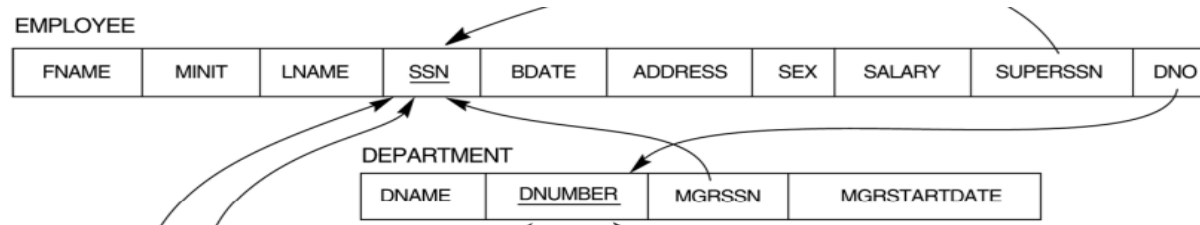
- Similar to a SELECT-PROJECT pair of relational algebra operations; the SELECT-clause specifies the *projection attributes* and the WHERE-clause specifies the *selection condition*
- However, the result of the query *may contain* duplicate tuples



Retrieve Queries in SQL (3)

- Query 6: Retrieve the name and address of all employees who work for the 'Research' department.

**Q6: SELECT FNAME, MINIT, LNAME, ADDRESS
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNAME='Research' AND DNUMBER=DNO**



- Similar to a SELECT-PROJECT-JOIN sequence of relational algebra operations
- (DNAME='Research') is a *selection condition* (corresponds to a SELECT operation in relational algebra)
- (DNUMBER=DNO) is a *join condition* (corresponds to a JOIN operation in relational algebra)

Retrieve Queries in SQL (4)

- Query 7: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

**Q7: SELECT PNUMBER, DNUM, LNAME, BDATE, ADDRESS
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE DNUM=DNUMBER AND MGRSSN=SSN AND
 PLOCATION='Stafford'**

- In Q5, there are *two* join conditions
- The join condition DNUM=DNUMBER relates a project to its controlling department
- The join condition MGRSSN=SSN relates the controlling department to the employee who manages that department

Alias

- In SQL, we can use the same name for two (or more) attributes as long as the attributes are in *different relations*

A query that refers to two or more attributes with the same name must *qualify* the attribute name with the relation name by *prefixing* the relation name to the attribute name

Example:

- EMPLOYEE.NAME, DEPARTMENT.NAME
- Means: EmployeeName, DepartmentName

Alias (2)

- Some queries **need to refer to the same relation twice**
- In this case, **aliases are given to the relation name**
- Query 8: For each employee, retrieve the employee's name, and the name of his or her immediate supervisor.

Q8: **SELECT** E.FNAME, E.LNAME, S.FNAME, S.LNAME
 FROM EMPLOYEE E, EMPLOYEE S
 WHERE E.SUPERSSN=S.SSN

- In Q8, the alternate relation names E and S are called *aliases* or *tuple variables* for the EMPLOYEE relation
- We can think of E and S as two *different copies* of EMPLOYEE; E represents employees in role of *supervisees* and S represents employees in role of *supervisors*

Alias (2)

- Aliasing can also be used in any SQL query for convenience
Can also use the **AS** keyword to specify aliases

Q9: **SELECT** E.FNAME, E.LNAME, S.FNAME AS FIRSTNAME
 FROM , S.LNAME AS LASTNAME
 WHERE EMPLOYEE AS E, EMPLOYEE AS S
 E.SUPERSSN=S.SSN

Distinct

- SQL does **not treat a relation as a set**; *duplicate tuples can appear*
- **To eliminate duplicate** tuples in a query result, the keyword **DISTINCT** is used
- For example, the result of Q10 may have duplicate SALARY values whereas Q10A does not have any duplicate values

**Q10: SELECT SALARY
 FROM EMPLOYEE**

**Q10A: SELECT DISTINCT SALARY
 FROM EMPLOYEE**

Union

- There is a **union operation (UNION)**, and in *some versions* of SQL there are set **difference (MINUS)** and **intersection (INTERSECT)** operations
- The resulting relations of these set operations are sets of tuples; *duplicate tuples are eliminated from the result*
- The set operations apply only to *union compatible relations* ; the two relations **must have the same attributes and the attributes must appear in the same order**

Union (2)

- Query 11: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
Q11:(SELECT PNAME
      FROM      PROJECT, DEPARTMENT, EMPLOYEE
      WHERE      DNUM=DNUMBER AND MGRSSN=SSN
      AND        LNAME='Smith')
UNION
(SELECT PNAME
      FROM      PROJECT, WORKS_ON, EMPLOYEE
      WHERE      PNUMBER=PNO AND ESSN=SSN
      AND        LNAME='Smith')
```

Nesting Queries Using “IN”

- A complete SELECT query, called a *nested query* , **can be specified within the WHERE-clause of another query**, called the *outer query*
- Many of the previous queries can be specified in an alternative form using nesting
- Query 12: Retrieve the name and address of all employees who work for the 'Research' department.

**Q12: SELECT FNAME, LNAME, ADDRESS
 FROM EMPLOYEE
 WHERE DNO IN (SELECT DNUMBER
 FROM DEPARTMENT
 WHERE DNAME='Research')**

Nesting Queries Using “IN” (2)

- The nested query *selects the number of the 'Research' department*
- The *outer query select an EMPLOYEE tuple* if its DNO value is in the result of either nested query
- The comparison operator **IN** compares a value *v* with a set (or multi-set) of values *V*, and evaluates to **TRUE** if *v* is one of the elements in *V*
- In general, we can have **several levels of nested queries**
- A reference to an *unqualified attribute* refers to the relation declared in the *innermost nested query*
- In this example, the nested query is *not correlated* with the outer query



- Q13: SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E
WHERE E.SSN IN (SELECT ESSN
FROM DEPENDENT
WHERE ESSN=E.SSN AND
E.FNAME=DEPENDENT_NAME)**

Exist Function

- **EXISTS** is used to check whether the result of a correlated nested query is empty (contains no tuples) or not
- We can formulate Query 14 in an alternative form that uses EXISTS as Q12B below
- Query 14: Retrieve the name of each employee who has a dependent with the same first name as the employee.

**Q14: SELECT FNAME, LNAME
 FROM EMPLOYEE
 WHERE EXISTS (SELECT *
 FROM DEPENDENT
 WHERE SSN=ESSN AND
 FNAME=DEPENDENT_NAME)**

IS NULL / IS NOT

- SQL allows queries that **check if a value is NULL** (missing or undefined or not applicable)
- SQL uses **IS or IS NOT** to compare NULLs because it considers each NULL value distinct from other NULL values, so equality comparison is not appropriate.

- Query 15: Retrieve the names of all employees who do not have supervisors.

**Q15: SELECT FNAME, LNAME
 FROM EMPLOYEE
 WHERE SUPERSSN **IS NULL****

Note: If a join condition is specified, tuples with NULL values for the join attributes are not included in the result

LIKE

- The **LIKE** comparison operator is used to compare partial strings
- Two reserved characters are used: '%' (or '*' in some implementations) replaces an arbitrary number of characters, and '_' replaces a single arbitrary character
- Query 16: Retrieve all employees whose address is in Houston, Texas. Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX'.

```
Q16:  SELECT  FNAME, LNAME  
        FROM    EMPLOYEE  
        WHERE   ADDRESS LIKE  
              '%Houston,TX%'
```

LIKE(2)

- The **LIKE** comparison operator is used to compare partial strings
- Two reserved characters are used: '%' (or '*' in some implementations) replaces an arbitrary number of characters, and '_' replaces a single arbitrary character
- Query 16: Retrieve all employees whose address is in Houston, Texas. Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX'.

**Q16: SELECT FNAME, LNAME
 FROM EMPLOYEE
 WHERE ADDRESS LIKE
 '%Houston,TX%'**

LIKE(3)

- Query 17: Retrieve all employees who were born during the 1950s. Here, '5' must be the 8th character of the string (according to our format for date), so the BDATE value is '_____5_', with each underscore as a place holder for a single arbitrary character.

Q17 **: SELECT FNAME, LNAME**
 FROM EMPLOYEE
 WHERE BDATE LIKE '_____5_'

- The LIKE operator allows us to get around the fact that each value is considered atomic and indivisible; hence, in SQL, character string attribute values are not atomic

ARITHMETIC OPERATIONS

- The standard arithmetic operators '+', '-', '*', and '/' (for addition, subtraction, multiplication, and division, respectively) can be applied to numeric values in an SQL query result
- Query 18: Show the effect of giving all employees who work on the 'ProductX' project a 10% raise.

```
Q18:SELECT  FNAME, LNAME, 1.1*SALARY
          FROM      EMPLOYEE, WORKS_ON, PROJECT
          WHERE      SSN=ESSN AND PNO=PNUMBER AND
          PNAME='ProductX'
```

AGGREGATE FUNCTION

- Include **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**
- Query 19: Find the maximum salary, the minimum salary, and the average salary among all employees.

**Q19: SELECT MAX(SALARY), MIN(SALARY), AVG(SALARY)
 FROM EMPLOYEE**

- Some SQL implementations *may not allow more than one function* in the SELECT-clause

AGGREGATE FUNCTION (2)

- Query 20: Find number of employees who work for the 'Research' department.

Q20: **SELECT COUNT(*)**
 FROMEMPLOYEE, DEPARTMENT
 WHERE DNO=DNUMBER AND DNAME='Research'

- Query 21: Find the maximum salary, the minimum salary, and the average salary among employees who work for the 'Research' department.

Q21: SELECT MAX(SALARY), MIN(SALARY),
 AVG(SALARY)
 FROMEMPLOYEE, DEPARTMENT
 WHERE DNO=DNUMBER AND
 DNAME='Research'

Grouping

- In many cases, we want to apply the aggregate functions to *subgroups of tuples in a relation*
- Each subgroup of tuples consists of the set of tuples that have *the same value* for the *grouping attribute(s)*
- The function is applied to each subgroup independently
- SQL has a **GROUP BY**-clause for specifying the grouping attributes, which *must also appear in the SELECT-clause*

Grouping (2)

- Query 22: For each department, retrieve the department number, the number of employees in the department, and their average salary.

**Q22:SELECT DNO, COUNT (*), AVG (SALARY)
FROMEMPLOYEE
GROUP BY DNO**

- In Q20, the EMPLOYEE tuples are divided into groups--each group having the same value for the grouping attribute DNO
- The COUNT and AVG functions are applied to each such group of tuples separately
- The SELECT-clause includes only the grouping attribute and the functions to be applied on each group of tuples
- A join condition can be used in conjunction with grouping

Having Clause

- Sometimes we want to retrieve the values of these functions for only those *groups that satisfy certain conditions*
- The HAVING-clause is used for specifying a selection condition on groups (rather than on individual tuples)

Query 23: For each project *on which more than two employees work* , retrieve the project number, project name, and the number of employees who work on that project.

Q23:

```
SELECT  PNUMBER, PNAME, COUNT (*)
FROM    PROJECT, WORKS_ON
WHERE   PNUMBER=PNO
GROUP BY PNUMBER, PNAME
HAVING  COUNT (*) > 2
```

Order By

- The **ORDER BY** clause is used to sort the tuples in a query result based on the values of some attribute(s)
- Query 24: Retrieve a list of employees and the projects each works in, ordered by the employee's department, and within each department ordered alphabetically by employee last name.

Q24: **SELECT DNAME, LNAME, FNAME, PNAME**
 FROM DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT
 WHERE DNUMBER=DNO AND SSN=ESSN
 AND PNO=PNUMBER
 ORDER BY DNAME, LNAME

Join

- Can specify a "joined relation" in the FROM-clause
- Looks like any other relation but is the result of a join
- Allows the user to specify different types of joins (regular "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, etc)
- Examples:

**Q25: SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
 FROM EMPLOYEE E, EMPLOYEE S
 WHERE E.SUPERSSN=S.SSN**

can be written as:

**Q25: SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
 FROM EMPLOYEE E
 LEFT OUTER JOIN EMPLOYEES ON E.SUPERSSN=S.SSN**

Join (2)

Q26: **SELECT** **FNAME, LNAME, ADDRESS**
 FROM **EMPLOYEE, DEPARTMENT**
 WHERE **DNAME='Research' AND DNUMBER=DNO**

could be written as:

Q26 : **SELECT** **FNAME, LNAME, ADDRESS**
 FROM **EMPLOYEE**
 JOIN DEPARTMENT ON DNUMBER=DNO
 WHERE **DNAME='Research'**

INSERT

- In its simplest form, it is used to add one or more tuples to a relation
- Attribute values should be listed in the same order as the attributes were specified in the CREATE TABLE command

Example:

U1: INSERT INTO EMPLOYEE

**VALUES ('Richard', 'Marini', '653298653', '30-DEC-52',
'98 Oak Forest, Katy, TX', 'M', 37000, '987654321', 4)**

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple
- Attributes with NULL values can be left out
- Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

**U1A: INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marini', '653298653')**

INSERT (2)

- Example: Suppose we want to create a temporary table that has the name, number of employees, and total salaries for each department. A table DEPTS_INFO is created by U3A, and is loaded with the summary information retrieved from the database by the query in U3B.

U2A: **CREATE TABLE DEPTS_INFO**
 (DEPT_NAME VARCHAR(10),
 NO_OF_EMPS INTEGER,
 TOTAL_SAL INTEGER);

U2B: **INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL_SAL)**
 SELECT DNAME, COUNT (*), SUM (SALARY)
 FROM DEPARTMENT, EMPLOYEE
 WHERE DNUMBER=DNO
 GROUP BY DNAME ;

DELETE

- Removes tuples from a relation
- Includes a WHERE-clause to select the tuples to be deleted
- Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
- A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
- The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause
- Referential integrity should be enforced

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity should be enforced
- Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

U4: UPDATE PROJECT

SET PLOCATION = 'Bellaire', DNUM = 5

WHERE PNUMBER=10

View

- A view is a “virtual” table that is derived from other tables
- Allows for limited update operations (since the table may not physically be stored)
- Allows full query operations
- A convenience for expressing certain operations
- SQL command: `CREATE VIEW`
 - a table (view) name
 - a possible list of attribute names (for example, when arithmetic operations are specified or when we want the names to be different from the attributes in the base relations)
 - a query to specify the table contents

View (2)

Specify a different WORKS_ON table

```
CREATE VIEW WORKS_ON_NEW AS  
SELECT FNAME, LNAME, PNAME, HOURS  
FROM EMPLOYEE, PROJECT, WORKS_ON  
WHERE SSN=ESSN AND PNO=PNUMBER  
GROUP BY PNAME;
```



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,
niscaya ilmu yang sedikit pun akan bermanfaat baginya